Movie Recommendation Hub: An AI-Powered, Microservices-Based Platform on Self-Hosted Kubernetes

PSIA 2026/2027 – Project Report – **Group 17**

Manuel Fernandes 106410 João Russo 116543 Tiago Fonseca 112346

Instituto Superior Técnico, Universidade de Lisboa, Lisbon, Portugal

Video: `https://youtu.be/Drgl3k404wE` | **Repository:** `https://gitlab.rnl.tecnico.ulisboa.pt/psia/groups26/g17`

1 INTRODUCTION

The PSIA project asks for a realistic, tiered, microservices-based application with AI components, deployed on a public cloud with Infrastructure as Code (IaC) end to end, where Terraform creates the virtual machines, Ansible configures them, and a *self-hosted* Kubernetes cluster runs the application (managed offerings such as GKE are not allowed). The system must be monitored, and extended with one advanced feature.

Our application is a *Movie Recommendation Hub*. Users register and log in through Keycloak (OpenID Connect [7]), keep a profile with their favourite genres, watched and saved movies and the users they follow, browse and search about 45 000 movies from The Movies Dataset [1], and view other users' profiles. A language model served by Ollama recommends movies in the two ways the statement requires, *on demand* from a request the user types ("a horror movie from the 80s") and *proactively* through a background agent that prepares and caches a set per profile, so users see fresh recommendations as soon as they log in. Each backend service owns its own PostgreSQL database [6].

The system runs on Google Cloud (`europe-west4`) on a kubeadm cluster of three VMs, monitored with Prometheus [4] and Grafana. As advanced feature we chose *persistent state*, so all VMs can be stopped and restarted, or destroyed and recreated, without losing any application state. The demonstration video is at `https://youtu.be/Drgl3k404wE`, and the code with full deployment instructions is at `https://gitlab.rnl.tecnico.ulisboa.pt/psia/groups26/g17`. Section 2 presents the architecture and deployment, Section 3 the persistent state, and Section 4 a throughput and latency study.

2 SYSTEM DESIGN AND ARCHITECTURE

The system follows the tiered architecture of the specification (Fig. 1, Table 1). The *frontend* is a Next.js web application. In the cluster an NGINX reverse proxy is the only public entry point. It terminates TLS and routes `/` to the frontend, `/api/*` to the services, `/auth/` to Keycloak and `/grafana/` to Grafana. The *backend* has four Quarkus services, namely the Movies API (catalogue, search, search by meaning), the Profiles API (genres, watched and saved movies, follows), the Recommendation API, which stores recommendations and queues the work, and the recommendation worker, which takes that work from *Apache Kafka* and queries the *inference engine*, Ollama. The *data tier* is PostgreSQL 16, one instance per service plus one for Keycloak, and services read each other's data only through their APIs. Prometheus and Grafana form the *monitoring server*.

Table 1: Components and technologies.

Component	Technology
Frontend	Next.js 16, React 19, Tailwind 4, keycloak-js
Identity provider	Keycloak 26 (OIDC, PKCE)
Four services	Quarkus 3.39 (Java 21+), Hibernate Panache, Flyway, Micrometer
Inference engine	Ollama on CPU, Qwen2.5 3B, nomic-embed-text
Message queue	Apache Kafka 4.3 (Strimzi, 3 brokers, KRaft)
Databases	PostgreSQL 16 per service (+ pgvector)
Local deployment	Docker Compose, `just`
Cloud deployment	Terraform, Ansible, kubeadm, Calico, NGINX
Monitoring	Prometheus, Grafana (kube-prometheus-stack)

The browser obtains a short-lived JWT from Keycloak (authorization code flow with PKCE) and sends it to every API, which checks its signature, issuer and its own audience, so tokens issued for other clients are rejected. The recommendation service and the worker call other services as themselves, with Keycloak service-account tokens (client credentials).

2.1 Backend Services, Data and AI

Each service is a Quarkus application whose schema Flyway owns. The catalogue comes from The Movies Dataset, whose `movies_metadata.csv` was cleaned (column-shifted rows and duplicate ids dropped, genre lists parsed) into 45 430 movies and 20 genres, shipped as a `pg_dump`. Profiles store only movie and genre ids, with no foreign keys across databases, so the Profiles API checks ids through the Movies API before storing them. A profile is created on the user's first authenticated request, from the token's subject and username.

Search. `GET /movies/search` uses PostgreSQL full-text search over weighted, accent-insensitive title and description vectors, prefix matching on title words, and a trigram fallback for typos, ranked in four tiers (title match, title prefix, description, similar title). For Ask AI, the Movies API also searches *by meaning*. A background job stores a 768-dimensional `nomic-embed-text` vector for each of the ∼9 100 movies with at least 50 votes in pgvector, and a request is ranked by vector similarity.

Recommendations. A set of ten recommendations takes the model minutes on a CPU, so the work is asynchronous. The Recommendation API answers at once (202) and puts a job, with a snapshot of the profile, on the Kafka topic `recommendation.requests` (6 partitions, keyed by user). The worker takes one job at a time and publishes the result on `recommendation.results`. The API stores it, and a third topic, `recommendation.ready`, lets every

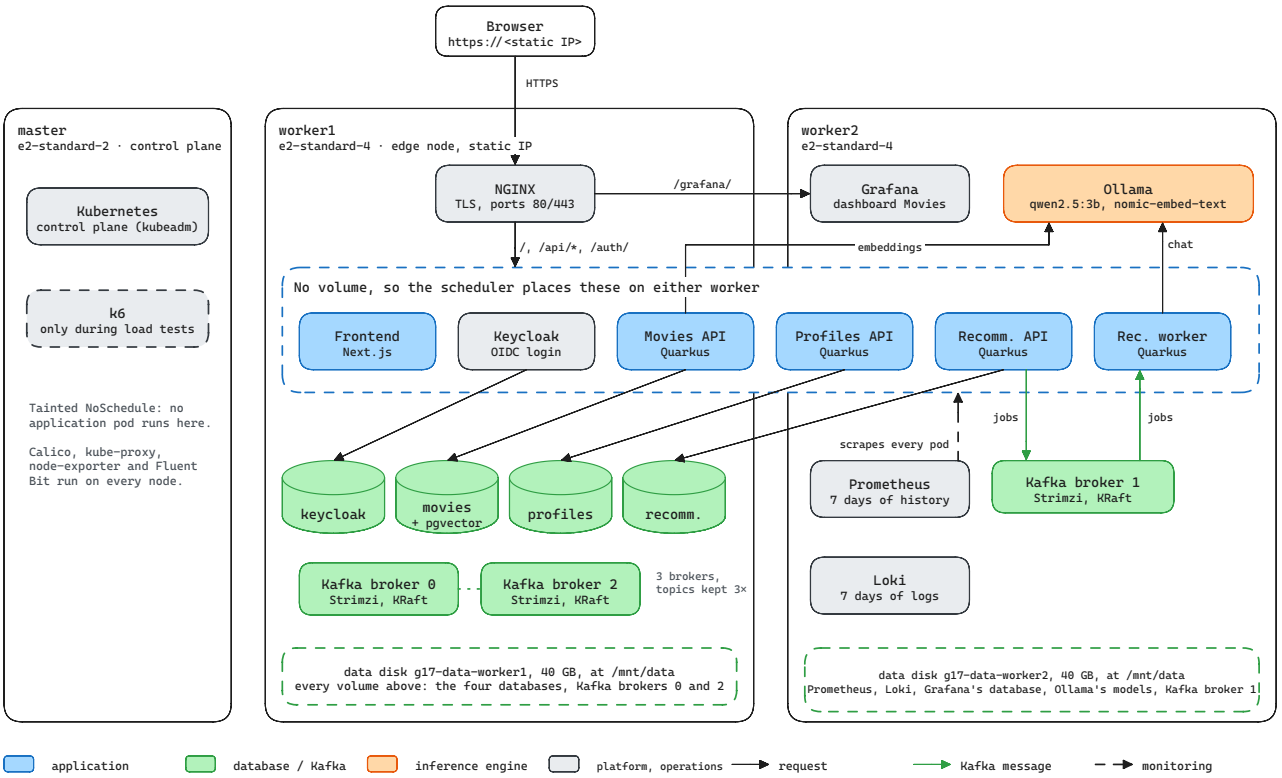


Figure 1: The deployment on Google Cloud, with the three VMs, the pods on each, and the workers' data disks. Pods with a volume (databases, Kafka brokers, Ollama, Prometheus, Grafana, Loki) always run on the worker that holds their data disk, while the stateless ones can run on either worker.

API instance notify its connected browsers through server-sent events. A job leaves the queue only once it is done, so a worker that stops mid-job leaves it for the next one. There are two kinds of request, *Recommended for you*, made from the profile and what followed users watched, and *Ask AI*, made from free text. The worker narrows the 45 000 movies first. It turns the request into filters (genres, years, keywords), the Movies API returns 30 candidates (closest in meaning, then best rated) and up to 20 come from followed users, and the model chooses ten. Its answer must follow a JSON schema that only allows the candidates' ids, so it cannot invent movies, and if it chooses fewer than ten, the best-ranked candidates fill the gaps.

A *background agent* runs every 10 minutes and keeps *Recommended for you* current without redoing work. Each set records a fingerprint of the inputs it was made from, and the agent queues a new set only for profiles whose inputs changed (after 2 quiet minutes), at most 20 per run, retrying failures with exponential back-off. Identical inputs share results, and a request still pending after an hour is failed by a sweeper as a safety net.

Model choice. The statement asks for an existing model, not a trained one, that gives acceptable recommendations. We chose Qwen2.5 3B [5] in Ollama because it fits the workers' memory with room for the rest of the system (about 2.5 GB), runs on their CPUs, and follows Ollama's JSON-schema output, which our pipeline relies on to keep answers inside the candidate list. Smaller variants (0.5B, 1.5B) answer faster but choose less well, and a larger model would make each recommendation slower still on these CPUs. Narrowing the catalogue to 50 candidates before the model sees it keeps the prompt near 2 000 tokens, which is what makes a 3B model usable. `nomic-embed-text` [3] is a small embedding model (about 270 MB) that the same Ollama server runs. We did not benchmark alternatives such as other 2–4B instruct models, and a GPU node would change this choice.

Security and testing. Keycloak hosts a `movies` realm, imported from JSON, with self-registration, brute-force protection and two public clients (the frontend and the Swagger UIs). Each API exposes health checks, Prometheus metrics and OpenAPI. Tests run against a throwaway PostgreSQL through Quarkus Dev Services, with injected identities instead of Keycloak and in-memory channels instead of Kafka, while the worker's tests replace the model with a mock.

2.2 Frontend

The web interface is a Next.js 16 application in TypeScript. Its pages render in the browser and call the APIs directly. The pages are `/movies` (catalogue with search, sorting and infinite scrolling, topped by *Recommended for you*), `/movies/[id]` (details, mark as watched, watch later), `/recommendations` (*Ask AI* and the last five answers), `/users` and `/users/[id]` (profiles, follows). An `AuthGate` logs users in with `keycloak-js`, keeps the token only in memory and refreshes it before each call. Because recommendations take minutes, the interface never waits for them, and a fetch-based server-sent-events stream to the Recommendation API notifies the user when a result is ready. In the cluster the image uses relative addresses (`/api`, `/auth`), so one image works at any address.

2.3 Deployment and Operations

Local. The whole system runs in Docker as three Compose projects on one network, one for the infrastructure (databases, Kafka, Ollama and a job that downloads the models, Keycloak), one for the four services and one for the frontend. One multi-stage Dockerfile builds any service, and the same images serve the cloud. A `justfile` starts the projects in order and waits for each to be healthy, since Compose cannot order separate projects.

Kubernetes on GCP. Everything is driven from a management container holding Terraform, Ansible, `kubectl` and `gcloud`. Terraform creates, in a separate stack that outlives the VMs, one data disk per worker and a static IP (Section 3), and then a VPC, firewall rules and three Ubuntu 24.04 VMs in `europe-west4-b`, a control plane (`e2-standard-2`) and two workers (`e2-standard-4`). Four Ansible playbooks then prepare the nodes (containerd, kubeadm), create the cluster with Calico as pod network, install the platform with Helm (the Strimzi Kafka operator, kube-prometheus-stack and the volumes), and deploy the application from manifest templates, with a PostgreSQL StatefulSet per database with a metrics sidecar, Kafka with three brokers and topics replicated three times, Ollama, Keycloak, a Deployment per service with health probes, and NetworkPolicies so that each database accepts only its own service. NGINX runs on the worker that owns the static IP, and every other port stays closed to the Internet. Secrets are generated on the first run and kept out of git, and every step can be re-run without changing what is already right. A deployment from nothing takes about 25 minutes:

```
just images docker.io/<user> v1 # push images
just mgmt-start && just mgmt    # mgmt shell
just all              # VMs, cluster, app
just stop | start | destroy
```

The repository's `README.md` and `infra/README.md` give every step, including the one-time Google Cloud setup.

Monitoring. Prometheus scrapes, every 15 s, each service's Micrometer metrics, the databases' and NGINX's exporters, Kafka's lag and the nodes. Beyond the standard HTTP and JVM metrics, each service records application metrics, such as searches and whether they found anything, profile changes, recommendation requests by kind and outcome (including those answered without the model), the time from a request to its answer, jobs waiting in Kafka, and each model call's duration and tokens, since Ollama has no metrics of its own. A provisioned Grafana dashboard, *Movies*, is built on them, and is served at `/grafana`, while Prometheus itself is not public. Fluent Bit, on every node, collects each container's log, and Loki aggregates them for 7 days, so they can be searched in Grafana across pods and restarts. Prometheus also evaluates our alert rules (Table 2), besides kube-prometheus-stack's defaults, and Grafana lists them by state. There is no Alertmanager, so alerts are not sent anywhere.

3 ADVANCED COMPONENT: PERSISTENT STATE

Goal. All VMs can be stopped and started again, or destroyed altogether (`terraform destroy`) and recreated days later as a new Kubernetes cluster. In both cases every user, profile, follow, watched movie and stored recommendation must still be there, with no manual steps, while a first deployment with no data starts empty through the same commands. The data stays on disks attached to the VMs, so the storage is self-hosted rather than an external managed service, and recovery does not depend on keeping the Terraform state. Table 3 lists the state to keep, since the APIs, the worker, the frontend and NGINX are stateless.

Table 2: Our alert rules, besides kube-prometheus-stack's defaults.

Alert	Severity	Fires when
`ServiceDown`	critical	A Service in `movies` can't be scraped for 2 min
`HighErrorRate`	warning	A service answers more than 6 server errors (5xx) a minute, for 5 min
`RecommendationsFailing`	warning	The worker failed more than 2 recommendations in 30 min
`RecommendationQueueStuck`	warning	More than 20 jobs wait for the worker for 15 min
`PodRestarting`	warning	A pod in `movies` restarted more than twice in 15 min

Table 3: Stateful components and where their state lives. All volumes are folders on the workers' data disks.

State	Mechanism
Four PostgreSQL databases (catalogue and embeddings, profiles, recommendations, Keycloak's realm and users)	StatefulSet, local PV, worker1
Kafka (queued jobs, offsets)	3 brokers, local PV each, cluster ID restored
Ollama's models; Prometheus history; Grafana's database	local PVs, worker2
Public address (named in the realm)	static IP, separate stack
Kubernetes (etcd, certificates)	boot disk; rebuilt after destroy

Storage choice. Each database is a one-replica StatefulSet whose claim binds to a PersistentVolume [8]. We back the volumes with *local* PVs on a Compute Engine data disk per worker, which adds no components to small VMs. Our first version used local-path-provisioner on the boot disks, which survived stopping the VMs but not `terraform destroy`, which deletes boot disks, nor a new cluster, where it would create new empty folders. Longhorn [9] would replicate volumes across nodes at the cost of an extra storage system on every node. Rook-Ceph needs far more resources, an NFS VM is a single point of failure and fragile under PostgreSQL, and the GCE CSI driver would let pods follow their disks between nodes but needs cloud credentials in the cluster. The trade-off we accept is that each pod is pinned to its worker and its data has no replica.

Design. (i) *Data outlives the VMs.* The data disks and the static IP belong to a second Terraform stack with its own state, which the VM stack only looks up by name, so destroying the VMs cannot delete them, and only an explicit `just destroy-data` does. Before applying, `just persistent` imports any disk or address that exists in the project but not in its state, so the first deployment creates them and later ones reuse them. A worker formats

its disk only if it is blank, and kubelet starts only once it is mounted. (ii) *Volumes are found again by name.* There is no dynamic provisioner, so the platform playbook declares one PV per claim, with `Retain`, a `claimRef` to the claim's namespace and name, a node affinity and a fixed folder, so the same claim in a new cluster binds to the same data. (iii) *Initialisation is idempotent.* PostgreSQL runs `initdb` only on an empty folder, Flyway applies only pending migrations, the playbook loads the catalogue only once, Keycloak imports its realm only if absent, and Ollama downloads only missing models. Kafka needs one more step, since Strimzi gives a new cluster a new ID that brokers refuse, so the playbook reads the old ID from the disk and gives it back to the new Kafka resource, following Strimzi's recovery procedure. (iv) *Results live in PostgreSQL* and pending jobs in Kafka. (v) *Stop and start.* `just stop` and `just start` change one Terraform variable, which Google's provider turns into Compute Engine stop and start calls. Stopped VMs cost only their disks and the static IP. Since stopped VMs lose their ephemeral addresses, `start` rereads them, rewrites Ansible's inventory, points the kubeconfig at the new master address, and waits until the application answers.

Validation. A snapshot script records each database's row counts, the finished recommendations, Keycloak's users, Kafka's topics and consumer offsets, the models, Prometheus's oldest sample, Grafana's users and the volumes, and a compare script diffs two snapshots. After a full stop and start, the snapshots were identical, existing users logged in, and their data and recommendations were served. The application answered 239 s after `just start`, and `just stop` takes about 1.5 minutes. A recommendation being computed when the VMs stopped failed and its user was told so, while queued ones were processed afterwards. We then destroyed every VM and redeployed on the same disks, with marker rows in each database and messages in a Kafka topic, and the snapshots were identical, every marker was there, the consumer group resumed where it stopped, and Prometheus returned the same historical values. Rebuilding took about 25 minutes, and while destroyed the disks and address cost about 15 USD a month. The limits are those of local volumes, since the data survives losing the VMs but not losing a data disk.

4 EVALUATION

Method. We measured throughput and latency as the number of concurrent clients grows, with k6 [2] running in a pod on the control-plane node, which runs no application pods, so it shares no CPU with the system it measures and adds no wide-area latency. Each client logs in as its own test user through a temporary Keycloak client and then loops, sending one action from a weighted mix and waiting 0.5–1.5 s (1 s on average). The mix is browsing the catalogue (35%), search (25%), a movie page (10%), a profile and its watched movies (15%), marking and unmarking a movie as watched (5%) and reading *Recommended for you* from the cache (10%), about 1.2 requests per action, all through NGINX's public address. On-demand recommendations are left out, because one takes the model about 4.5 minutes on these CPUs, which is why they are asynchronous. Each step ramps up for 30 s, warms up for 60 s and is measured for 180 s. Token requests and the warm-up are excluded, and each step ran three times. The cluster is that of Section 2.3, with one replica per service, and Prometheus recorded the CPU of every pod and node.

Table 4: Throughput and latency for the request mix (mean of three runs, errors ≤0.04% throughout).

Clients	req/s	p50 (ms)	p95 (ms)	p99 (ms)	worker1
1	1.2	12	23	66	11%
10	11.6	12	25	88	16%
50	58.9	11	31	105	34%
100	117.1	12	50	161	56%
200	215.9	28	291	890	97%
300	205.6	473	1 661	3 963	100%
400	198.8	928	3 254	5 831	—

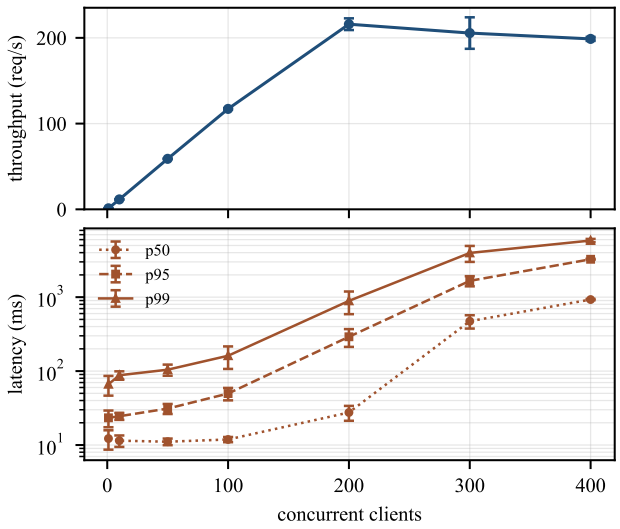


Figure 2: Throughput (top) and latency percentiles (bottom, log scale) as the number of concurrent clients grows.

Results. Table 4 and Fig. 2 show two regimes. Up to 100 clients the system is lightly loaded and throughput grows linearly, about 1.17 req/s per client, which is what a closed loop with a 1 s think time and short responses predicts (1.2 requests per second of think time), and the median latency stays near 12 ms. At 200 clients throughput reaches its peak, about 216 req/s, and latency starts to rise (p95 291 ms). Beyond that, adding clients adds no throughput, which stays near 200–206 req/s, while every percentile grows, and at 400 clients the median is 0.93 s and p99 5.8 s. The system slows down rather than fails, with at most 0.04% errors, and these were 500s from `GET /recommendations/me`, whose call to the Movies API for titles has no error handling, at low load as well as high.

Bottleneck. The limit is the CPU of worker1, at 97% with 200 clients and 100% with 300. The scheduler had placed every stateless Deployment on worker1, next to the four databases, so worker2 stayed at 8% and the cluster's second worker added almost nothing. The largest consumer is the movies database, which reached 2.3 of worker1's 4 vCPUs, because 70% of the actions are catalogue reads, and search runs full-text and trigram queries. The Movies API itself used about 0.5 cores. The master, running k6, never exceeded 38% CPU, so the generator did not limit the results. Spreading the stateless pods across workers (topology spread constraints), adding replicas, and caching or replicating the catalogue database are the direct remedies, while horizontal autoscaling, the advanced option we did not take, would add replicas only if they could be sched-

uled on the idle worker.

5 CONCLUSION

We built a Movie Recommendation Hub as four Quarkus microservices and a Next.js frontend, with Keycloak, one PostgreSQL database per service, and Kafka between the recommendation service and its worker. It runs locally in Docker with one command, and in Google Cloud on a self-hosted Kubernetes cluster created with Terraform and Ansible and monitored with Prometheus and Grafana. Qwen2.5 3B, served by Ollama, recommends movies on demand and, through a background agent, ahead of time. Data disks that outlive the VMs, volumes bound to them by name and idempotent initialisation let the whole infrastructure be stopped, or destroyed and recreated, without losing state, which we verified end to end. The cluster serves about 216 req/s before worker1's CPU saturates, mostly in the catalogue database. Next steps would be spreading and replicating the stateless services, a GPU node for the model, and a CI/CD pipeline.

DECLARATION OF USE OF AI

We used Claude (Anthropic), through Claude Code, as a coding assistant while developing and deploying the system, and to help write and revise its documentation, diagrams and this report. The group reviewed, tested and edited all of its output, and takes full responsibility for the content.

REFERENCES

[1] Rounak Banik. The movies dataset. Kaggle, https://www.kaggle.com/datasets/rounakbanik/the-movies-dataset, 2017.

[2] Grafana Labs. Grafana k6 documentation. https://grafana.com/docs/k6/latest/. Accessed October 2026.

[3] Zach Nussbaum, John X. Morris, Brandon Duderstadt, and Andriy Mulyar. Nomic embed: Training a reproducible long context text embedder. arXiv:2402.01613, https://arxiv.org/abs/2402.01613, 2024.

[4] Prometheus Authors. Prometheus: Monitoring system and time series database. https://prometheus.io/docs/introduction/overview/. Accessed October 2026.

[5] Qwen Team. Qwen2.5 technical report. arXiv:2412.15115, https://arxiv.org/abs/2412.15115, 2024.

[6] Chris Richardson. Pattern: Database per service. https://microservices.io/patterns/data/database-per-service.html. Accessed October 2026.

[7] Nat Sakimura, John Bradley, Michael B. Jones, Breno de Medeiros, and Chuck Mortimore. OpenID Connect Core 1.0. OpenID Foundation, https://openid.net/specs/openid-connect-core-1_0.html, 2014.

[8] The Kubernetes Authors. Persistent volumes. Kubernetes documentation, https://kubernetes.io/docs/concepts/storage/persistent-volumes/. Accessed October 2026.

[9] The Longhorn Authors. Longhorn: Cloud native distributed block storage for Kubernetes. https://longhorn.io/docs/. Accessed October 2026.